

WeatherWise

A full-stack weather aggregator that combines multiple forecast sources into a single consensus reading, tracks prediction accuracy over time, and surfaces honest uncertainty to the user.

Features

Core Consensus Engine

- Aggregates **Open-Meteo**, **OpenWeatherMap**, **Tomorrow.io**, and **WeatherAPI** into a single weighted consensus
- Computes a **confidence score** (0–100) based on inter-source agreement
- Shows a **dispute badge** when sources meaningfully disagree (configurable threshold, default 3 °C)
- Confidence bar colour-codes certainty at a glance
- Dynamic weights: sources with better rolling accuracy receive higher weight automatically

Forecast

- **7-day or 14-day** daily forecast (days 8–14 shown with explicit confidence tiers: high / medium / low / trend-only)
- **Hourly forecast** for the next 48 h with temperature + precipitation bars
- **Extended hourly** hook — up to 7 days of hourly data for deeper views
- ForecastChart (Recharts) with switchable daily/hourly view

Radar Map

- Animated precipitation radar powered by **Leaflet** + **RainViewer** free API
- Dark CartoDB basemap centred on the searched city
- Play / pause / manual scrubber across past radar frames

Precipitation Timeline

- 24-hour bar chart of rain probability (colour-coded by intensity: blue-100 → blue-400)
- Highlights peak probability and shows per-hour mm/hr on hover

Weekend Planner

- Dedicated 📅 view showing Saturday and Sunday side-by-side
- Each day card: high/low, rain %, UV index, wind gusts, confidence pill (high / some uncertainty / trend only)
- Hourly rain bars for each day when data is available

Commute Mode

- Configure **morning departure** and **evening return** times (persisted to localStorage)
- Shows conditions, rain probability, wind speed, and a one-line recommendation for each commute slot

Historical Comparison

- Yesterday's actual high/low + same date one year ago via **Open-Meteo historical API**
- Temp delta vs. today's forecast shown in red/blue

Saved Locations

- Pin any city with the ★ button in the header
- Saved cities appear as chips — click to switch instantly, × to remove
- Persisted to localStorage via `useSavedLocations` hook

Custom Alerts

- User-defined notification thresholds: **wind speed, rain probability, temperature low, source disagreement**
- Fires browser `Notification` API when conditions are met (deduped per session per city)
- Thresholds persist to localStorage

Daily Digest Email

- Subscribe with any email address to receive a morning summary at **6:00 AM daily**
- Server-side cron job (node-cron) builds a plain-text digest from live forecast data
- File-backed subscriber store (`data/digest_subscriptions.json`)
- Unsubscribe endpoint removes the record immediately

Dark / Light Theme

- Toggle between dark (default) and light mode with 🌞 / 🌙 button in the header
- Preference persisted to localStorage and applied via `html.classList`

AI Chat Drawer

- **Groq-powered** assistant (llama-3.3-70b-versatile) that answers questions about the current forecast
- Pre-built prompt chips for common questions
- Thumbs up/down feedback on AI answers
- Responses reference actual consensus data pulled live
- Rate limited to 10 requests per 60 seconds per user

Accuracy Tracking

- PostgreSQL table records daily predictions at fetch time
- Rolling accuracy score computed per source, displayed in an Accuracy Leaderboard
- Accuracy scores feed back into consensus weighting
- Nightly cron job (2:15 AM) computes actuals and updates source scores

Additional UI

- **Feels-like** temperature displayed inline with hero temperature
- **Sun arc** animation showing current position in the day
- **Air quality** index card
- **Alerts banner** for severe weather from source APIs
- **Location feedback** panel (thumbs up/down on forecast quality)

- **Share card** — copy a plain-text summary of current conditions
- **Smart summary** — one-line natural-language description of conditions
- **Stat tooltips** — rich detail on hover for each weather metric
- **Best time widget** — estimated best window to go outside
- Press **/** anywhere to focus the city search bar
- **Offline banner** when connection is lost

Tech Stack

Layer	Tech
Frontend	React 18, TypeScript, Vite, Tailwind CSS
Charts	Recharts
Map	Leaflet, RainViewer API
State / Data	TanStack Query (React Query)
Backend	Node.js, Express, TypeScript
Database	PostgreSQL (prediction accuracy tracking)
AI	Groq API (llama-3.3-70b-versatile)
Weather APIs	Open-Meteo (free, no key), OpenWeatherMap, Tomorrow.io, WeatherAPI
Email cron	node-cron + Nodemailer
Caching	node-cache (in-memory, configurable TTL)

API Endpoints

GET	/api/weather/current	Current conditions (consensus)
GET	/api/weather/forecast	Daily forecast (?days=7 or 14)
GET	/api/weather/hourly	Hourly forecast (?days=1-7)
GET	/api/weather/alerts	Severe weather alerts
GET	/api/weather/accuracy	Rolling accuracy leaderboard
GET	/api/weather/precipitation-timeline	24h hourly rain probability
GET	/api/weather/historical	Yesterday + last year high/low
GET	/api/weather/feedback-summary	Aggregated user feedback for a city
GET	/api/weather/geocode/search	City autocomplete / coordinate lookup
POST	/api/weather/ai-chat	Groq AI weather Q&A
POST	/api/weather/feedback	Submit a forecast quality vote
POST	/api/weather/ai-feedback	Rate an AI chat response (up/down)
POST	/api/weather/digest/subscribe	Subscribe to daily email digest
DELETE	/api/weather/digest/unsubscribe	Remove digest subscription

Project Structure

```

weatherwise/
├── client/                                React + Vite frontend (port 5173 in dev)
│   └── src/
│       ├── components/                  30+ UI components (radar, forecast, alerts, ...)
│       ├── hooks/                      Data fetching hooks (useWeatherConsensus,
useSavedLocations, ...)
│       └── types/                      Shared TypeScript types
└── server/                             Express backend (port 3001)
    └── src/
        ├── routes/                    weather.ts – all API route handlers
        ├── services/                  consensus.ts, openMeteo.ts, tomorrowIo.ts,
weatherApi.ts, openWeatherMap.ts,
groqDecision.ts, emailDigest.ts
        ├── cache/                    weatherCache.ts – in-memory node-cache wrapper
        ├── db/                      pool.ts – PostgreSQL connection pool
        └── jobs/                     digestCron.ts (6 AM), accuracyCron.ts (2:15 AM)

```

Environment Variables

Copy `.env.example` to `.env` in the project root (or `server/.env`) and fill in the values you need. Only **PORT** is truly required — everything else enables optional features.

```

# — Server —————
PORT=3001                                # Server listening port (default: 3001)

# Your deployed app's public URL – used by digest email links and internal cron
calls
# e.g. https://weather.yourdomain.com
APP_URL=

# — Weather API keys (all optional – Open-Meteo needs no key and is always on) –
OPENWEATHERMAP_API_KEY=                  # free: 1,000 calls/day – openweathermap.org
TOMORROW_IO_API_KEY=                     # free: 500 calls/day – tomorrow.io
WEATHERAPI_KEY=                          # free: 1M calls/month – weatherapi.com

# — AI chat —————
GROQ_API_KEY=                            # free tier – console.groq.com
                                           # AI chat returns 503 if this is not set

# — Database (optional – enables accuracy tracking + adaptive weights) —————
DATABASE_URL=postgresql://user:password@host:5432/weatherwise
DATABASE_SSL=false                       # set to true for hosted Postgres
                                           # (Railway, Render, Supabase, etc.)

# — Email digest (optional – digest emails logged to console if not set) —————
SMTP_HOST=
SMTP_PORT=587
SMTP_USER=
SMTP_PASS=
SMTP_SECURE=false                       # true to use TLS (port 465)

```

```
EMAIL_FROM=                                # defaults to SMTP_USER if blank

# — Tuning —
CACHE_TTL_SECONDS=600                      # how long to cache weather responses (seconds)
DISPUTE_THRESHOLD_CELSIUS=3                # min °C spread between sources to show dispute badge

# — Development only (never set in production) —
# NODE_TLS_REJECT_UNAUTHORIZED=0          # disable TLS validation for local self-signed certs
```

Feature availability without optional variables

Variable	Missing behaviour
OPENWEATHERMAP_API_KEY	OpenWeatherMap source skipped; consensus uses remaining sources
TOMORROW_IO_API_KEY	Tomorrow.io source skipped
WEATHERAPI_KEY	WeatherAPI source skipped
GROQ_API_KEY	AI chat endpoint returns 503
DATABASE_URL	Accuracy tracking disabled; all sources weighted equally
SMTP_*	Digest emails printed to server console instead of sent
APP_URL	Digest email links point to http://localhost:3001

Getting Started

Prerequisites

- **Node.js** 18 or later
- **npm** 9 or later
- **PostgreSQL** (optional — only needed for accuracy tracking)

Development

```
# Install all dependencies (client + server)
npm install

# Copy the example env file and fill in your keys
cp .env.example .env

# Start client and server in dev mode (runs concurrently)
npm run dev
# Client → http://localhost:5173 (Vite dev server, HMR enabled)
# Server → http://localhost:3001 (Express with tsx --watch)
# Vite proxies /api/* requests to the Express server automatically
```

Production build

```
# Compile client (Vite) + server (TypeScript)
npm run build

# Start the production server
npm run start

# Express serves the built SPA from client/dist/ and listens on PORT (default 3001)
```

Running tests

```
npm run test
```

Deployment

The project ships with a `render.yaml` for one-click deployment to **Render.com**.

1. Fork / push this repo to GitHub.
2. Create a new **Web Service** on Render, point it at your repo.
3. Render will detect `render.yaml` and provision the web service + a managed PostgreSQL database automatically.
4. Set the following environment variables in the Render dashboard (marked `sync: false` in `render.yaml`):
 - `APP_URL` — your Render service URL (e.g. `https://weatherwise.onrender.com`)
 - `OPENWEATHERMAP_API_KEY`, `TOMORROW_IO_API_KEY`, `WEATHERAPI_KEY`
 - `GROQ_API_KEY`
 - `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `EMAIL_FROM` (if you want digest emails)
5. `DATABASE_URL` and `DATABASE_SSL=true` are wired up automatically from the managed database.

The build command is `npm run build` and the start command is `npm run start`.